

HOTEL ROOM BOOKING SYSTEM

Python Project Report

A Simple Console-Based Hotel Room Management Application

1. Introduction

The Family Hotel Room Booking System is a simple Python-based application designed to manage basic hotel room booking activities. In a traditional hotel, room availability, customer details, booking information, and cancellation may be handled manually. Manual management can take more time and may lead to mistakes while checking room availability or calculating the bill. This project provides a simple computerized solution for these activities.

The application works through a menu-driven console interface. The user can view available rooms, book a suitable room, view the current booking, cancel the booking, and exit the application. The system also validates the mobile number and limits the booking duration to a maximum of 15 days.

2. Problem Statement

Managing hotel rooms manually can become difficult when there are multiple rooms and customers. It is necessary to know which rooms are available, what type of room is suitable for a particular number of members, how much the customer should pay, and whether a booking has been cancelled. A simple software system can organize these operations and reduce manual effort.

3. Aim of the Project

The aim of this project is to develop an easy-to-use hotel room booking application using basic Python programming concepts. The system should provide room information, suggest an appropriate room type based on the number of members, complete a booking, calculate the total bill, display booking information, and cancel a booking.

4. Objectives

- To display hotel room numbers, room types, prices, and availability.
- To allow customers to book an available room.
- To suggest a suitable room type based on the number of members.
- To validate the customer's 10-digit mobile number.
- To validate the number of days for the stay.
- To calculate the total room cost automatically.
- To store and display customer booking information.
- To change the room status from Available to Booked after successful booking.
- To change the room status back to Available after cancellation.
- To provide a simple menu-driven user interface.

5. Scope of the Project

The project is suitable for demonstrating basic hotel room management using Python. It can be used as a small academic mini project and can be extended into a larger hotel management application. The current version focuses mainly on room availability, one active booking, billing, and cancellation.

6. Technologies Used

- Programming Language: Python
- Development Type: Console-based application
- Data Structures: Dictionary and List
- Control Statements: if, elif, else
- Loops: while and for
- Input/Output: input() and print()
- Validation: len(), isdigit(), and conditional checks

7. System Features

7.1 View Rooms

The View Rooms option displays all room numbers along with their room type, price per day, and current status. The room status can be Available or Booked.

7.2 Book Room

The Book Room option collects the customer's name, phone number, number of members, room number, and number of days. The system first checks the phone number and then suggests a room type based on the number of members.

7.3 View Booking

The View Booking option displays the currently stored booking information, including customer name, phone number, number of members, room number, room type, number of days, and total bill.

7.4 Cancel Booking

The Cancel Booking option allows the customer to cancel the active booking by entering the assigned room number. After successful cancellation, the room status is changed to Available and the stored booking details are cleared.

7.5 Exit

The Exit option terminates the program using the break statement.

8. Room Information

The program initially contains seven rooms. Each room is represented using a dictionary entry. The room number is used as the key, while the value is a list containing room type, price, and status.

- Room 101 – Single – ₹1500 per day – Available
- Room 102 – Single – ₹1500 per day – Available
- Room 201 – Double – ₹2500 per day – Available

- Room 202 – Double – ₹2500 per day – Available
- Room 301 – Deluxe – ₹3500 per day – Available
- Room 302 – Double – ₹2500 per day – Available
- Room 303 – Deluxe – ₹4500 per day – Available

9. Room Suggestion Logic

The system automatically suggests a room type based on the number of members entered by the customer. This makes the booking process easier and helps the customer select an appropriate room.

- 1 to 2 members → Single Room
- 3 to 5 members → Double Room
- More than 5 members → Deluxe Room

10. Input Validation

Input validation is used to reduce incorrect data entry. The mobile number must contain exactly 10 digits and only numeric characters. The number of stay days must be between 1 and 15. The system also checks whether the selected room exists, matches the suggested room type, and is currently available.

11. Billing Calculation

The total bill is calculated using the room price and number of days. The formula used is:

Total Bill = Room Price × Number of Days

For example, if a Double room costs ₹2500 per day and the customer stays for 3 days, the total bill will be ₹7500.

12. Working Principle

When the program starts, a menu containing five options is displayed. The user selects an option by entering a number. If View Rooms is selected, the program displays room information. If Book Room is selected, customer information is collected, a suitable room is suggested, and the booking is stored. The selected room status changes to Booked and the total bill is calculated.

If View Booking is selected, the program displays the stored booking. If Cancel Booking is selected, the assigned room is checked, the room is made Available again, and the booking dictionary is cleared. The menu continues to appear until the user selects Exit.

13. Program Modules

- Room Management Module – Stores room number, type, price, and availability.
- Customer Module – Collects customer name, phone number, and number of members.
- Room Suggestion Module – Selects Single, Double, or Deluxe based on members.
- Booking Module – Confirms the room and stores booking details.
- Billing Module – Calculates the total cost of the stay.
- Booking Display Module – Shows current booking information.
- Cancellation Module – Cancels the booking and releases the room.
- Menu Module – Controls the complete application flow.

14. Data Structures Used

A dictionary named rooms is used to store room information. The room number is the dictionary key and the corresponding value is a list containing room type, price, and status. Another dictionary named booking stores the details of the current customer booking.

Dictionaries are useful in this project because room numbers and booking fields can be accessed easily using keys. Lists are used inside the rooms dictionary because each room contains multiple related values.

15. Advantages

- Simple and easy-to-understand interface.
- Reduces manual work in basic room management.
- Automatically calculates the room bill.
- Checks room availability before booking.
- Provides automatic room suggestions.
- Validates the mobile number and stay duration.
- Updates room status after booking and cancellation.
- Useful for learning Python programming concepts.

16. Limitations

- The current program stores only one active booking at a time.
- Booking information is not permanently stored after the program is closed.
- It does not use a database.
- It does not include online payment facilities.
- It does not include login or user authentication.
- It is a console application and does not have a graphical user interface.

17. Future Enhancements

The project can be improved by adding a database such as MySQL or SQLite to store customer and booking information permanently. A graphical user interface can be developed using Tkinter or another Python framework. Multiple customers can be supported by storing bookings in a list or database. Additional features such as check-in and check-out dates, tax calculation, discounts, payment options, customer login, room search, invoice generation, and admin management can also be added.

18. Sample Booking Flow

- Select option 2 – Book Room.
- Enter customer name.
- Enter a valid 10-digit phone number.
- Enter number of members.
- System suggests the suitable room type.
- Select an available room number.
- Enter number of days between 1 and 15.
- System calculates the total bill.

- Room status changes to Booked.
- Booking details can be viewed using option 3.

19. Conclusion

The Family Hotel Room Booking System is a simple and useful Python mini project that demonstrates the application of programming concepts to a real-world problem. It provides essential hotel room operations such as viewing rooms, booking, billing, viewing booking details, and cancellation.

This project is especially useful for beginners because it demonstrates dictionaries, lists, loops, conditional statements, input validation, and basic data processing in one application. Although the current version is simple, it provides a good foundation for developing a complete hotel management system with a database and graphical interface.

20. Complete Python Code

```
rooms = {
    101: ["Single", 1500, "Available"],
    102: ["Single", 1500, "Available"],
    201: ["Double", 2500, "Available"],
    202: ["Double", 2500, "Available"],
    301: ["Deluxe", 3500, "Available"],
    302: ["Double", 2500, "Available"],
    303: ["Deluxe", 4500, "Available"],
}

booking = {}

while True:

    print("\n===== FAMILY HOTEL ROOM BOOKING =====")
    print("1. View Rooms")
    print("2. Book Room")
    print("3. View Booking")
    print("4. Cancel Booking")
    print("5. Exit")

    choice = int(input("\nEnter your choice: "))

    if choice == 1:
        print("\nRoom No    Type        Price        Status")
        for room, details in rooms.items():
            print(room, "    ", details[0], "    ₹", details[1], "    ", details[2])

    elif choice == 2:
        name = input("\nEnter customer name: ")

        while True:
            phone = input("Enter phone number: ")

            if len(phone) == 10 and phone.isdigit():
                break
            else:
                print("Invalid phone number! Please enter 10 digit Mobile number.")

        members = int(input("Number of members for check-in: "))
```

```

if members <= 2:
    suggested_type = "Single"
elif members <= 5:
    suggested_type = "Double"
else:
    suggested_type = "Deluxe"

print("\nSuggested Room Type:", suggested_type)
print("\nAvailable", suggested_type, "Rooms:")

for room, details in rooms.items():
    if details[0] == suggested_type and details[2] == "Available":
        print(room, "- ₹", details[1])

room_no = int(input("Enter room number: "))

if room_no in rooms:
    if rooms[room_no][0] == suggested_type:
        if rooms[room_no][2] == "Available":

            while True:
                days = int(input("Enter number of days : "))

                if 1 <= days <= 15:
                    break
                else:
                    print("Invalid number of days! HOTEL cannot do our service not more than 15
days!.")

            price = rooms[room_no][1]
            total = price * days

            rooms[room_no][2] = "Booked"

            booking["name"] = name
            booking["phone"] = phone
            booking["members"] = members
            booking["room"] = room_no
            booking["room_type"] = suggested_type
            booking["days"] = days
            booking["total"] = total

            print("\nRoom booked successfully!")
            print("Room Type:", suggested_type)
            print("Total Bill: ₹", total)

        else:
            print("\nRoom is already booked!.")

    else:
        print("Please select a", suggested_type, "room.")

else:
    print("Invalid room number.")

elif choice == 3:
    if booking:
        print("\n===== BOOKING DETAILS =====")
        print("Customer Name:", booking["name"])
        print("Phone:", booking["phone"])

```

```

        print("Number of Members:", booking["members"])
        print("Room Number:", booking["room"])
        print("Room Type:", booking["room_type"])
        print("Number of Days:", booking["days"])
        print("Total Bill: ₹", booking["total"])

    else:
        print("No booking found.")

elif choice == 4:
    if booking:

        room_no = int(input("Enter the room number to cancel: "))

        if room_no == booking["room"]:

            print("\n===== CANCELLATION DETAILS =====")
            print("Customer Name:", booking["name"])
            print("Mobile Number:", booking["phone"])
            print("Room Number:", booking["room"])

            rooms[room_no][2] = "Available"
            booking.clear()

            print("\nRoom cancelled successfully.")
            print("Room", room_no, "is now available.")

        else:
            print("Invalid room number! Please enter your assigned room number.")

    else:
        print("No booking found.")

elif choice == 5:
    print("THANK YOU for using FAMILY HOTEL ROOM BOOKING!")
    break

else:
    print("Invalid choice")

```

21. Code explanation

1. Creating the room data

rooms = {

Creates a **dictionary** called rooms. A dictionary stores data in **key : value** format.

101: ["Single", 1500, "Available"],

Room 101 → Single → ₹1500 → Available

102: ["Single", 1500, "Available"],

201: ["Double", 2500, "Available"],

202: ["Double", 2500, "Available"],

301: ["Deluxe", 3500, "Available"],

302: ["Double", 2500, "Available"],

303: ["Deluxe", 4500, "Available"],

- 101 → room number
- "Single" → room type
- "1500" → room price per day
- "Available" → current room status

So Python understands this as:

```
}
```

Closes the rooms dictionary.

2. Creating the booking dictionary

```
booking = {}
```

Creates an **empty dictionary** called booking.

It will later store information about the current customer, such as:

- Name
- Phone number
- Members
- Room number
- Room type
- Days
- Total bill

3. Starting the program

```
while True:
```

Creates an **infinite loop**.

This means the hotel menu keeps appearing again and again until the user chooses **Exit**

4. Displaying the menu

```
print("\n===== FAMILY HOTEL ROOM BOOKING =====")
```

Displays the title.

`\n` means **start from a new line**.

```
print("1. View Rooms")
```

```
print("2. Book Room")
```

```
print("3. View Booking")
```

```
print("4. Cancel Booking")
```

```
print("5. Exit")
```

Displays the five available operations.

5. Getting the user's choice

```
choice = int(input("\nEnter your choice: "))
```

There are two important functions here:

input()

Takes input from the user.

int()

Converts the input from text/string into an integer.

For example:

Enter your choice: 2

The value stored in choice becomes:

```
choice = 2
```

6. Option 1 — View Rooms

```
if choice == 1:
```

Checks whether the user selected **1**.

If yes, the room list is displayed.

```
print("\nRoom No   Type   Price   Status")
```

Displays the headings for the room information.

for room, details in rooms.items():

This is a **for loop**.

.items() gets both:

- room number
- room details

For example:

```
room = 101
```

```
details = ["Single", 1500, "Available"]
```

Then it moves to room 102, 201, etc.

```
print(room, " ", details[0], " ₹", details[1], " ", details[2])
```

Displays the room information.

Remember that list positions start from **0**:

details[0] → Room Type

details[1] → Price

details[2] → Status

So:

details[0]

means "Single"

details[1]

means 1500

details[2]

means "Available"

7. Option 2 — Book Room

elif choice == 2:

Checks whether the user selected **2 — Book Room**.

Getting customer name

```
name = input("\nEnter customer name: ")
```

Asks the customer for their name.

8. Phone number validation

while True:

Starts a loop that keeps asking for the phone number until a valid number is entered.

```
phone = input("Enter phone number: ")
```

Gets the customer's phone number.

```
if len(phone) == 10 and phone.isdigit():
```

This checks **two conditions**.

```
len(phone) == 10
```

Checks whether the phone number has exactly 10 characters.

```
phone.isdigit()
```

Checks whether all characters are numbers.

and

Means both conditions must be true.

9. Getting number of members

```
members = int(input("Number of members for check-in: "))
```

Asks how many people will stay in the room.

int() converts the input into a number.

10. Room suggestion

```
if members <= 2:
```

Checks if there are **2 or fewer members**.

```
suggested_type = "Single"
```

If yes, the system recommends a **Single room**.

```
elif members <= 5:
```

If the first condition is false, this checks whether members are **5 or fewer**.

Therefore, 3–5 members get:

```
suggested_type = "Double"
```

else:

If the number of members is greater than 5:

```
suggested_type = "Deluxe"
```

So your logic is:

Members Suggested room

1-2	Single
3-5	Double
6+	Deluxe

11. Displaying suggested room

```
print("\nSuggested Room Type:", suggested_type)
```

Displays the room recommended by the system.

12. Finding available rooms

```
for room, details in rooms.items():
```

Again, the program goes through every room.

```
if details[0] == suggested_type and details[2] == "Available":
```

This is very important.

It checks two things:

First condition

```
details[0] == suggested_type
```

Second condition

```
details[2] == "Available"
```

Both conditions must be true.

Then:

```
print(room, "- ₹", details[1])
```

Displays the room number and price.

13. Selecting a room

```
room_no = int(input("Enter room number: "))
```

Asks the customer to select a room.

14. Checking whether the room exists

```
if room_no in rooms:
```

Checks whether the entered room number exists in the rooms dictionary.

15. Checking room type

if rooms[room_no][0] == suggested_type:

Checks whether the selected room's type matches the recommended type.

6. Checking room availability

if rooms[room_no][2] == "Available":

Checks whether the selected room is available.

If it is available, booking can continue.

17. Number of days validation

while True:

Starts another loop.

days = int(input("Enter number of days : "))

Asks how many days the customer wants to stay.

if 1 <= days <= 15:

18. Getting room price

price = rooms[room_no][1]

Gets the price of the selected room.

19. Calculating total bill

*total = price * days*

Calculates:

Total Bill = Room Price × Number of Days

20. Updating room status

rooms[room_no][2] = "Booked"

Changes the room status from:

Available

to:

Booked

This prevents another customer from booking the same room.

21. Storing booking details

```
booking["name"] = name
```

Stores the customer's name.

```
booking["phone"] = phone
```

Stores the customer's phone number.

```
booking["members"] = members
```

Stores the number of members.

```
booking["room"] = room_no
```

Stores the room number.

```
booking["room_type"] = suggested_type
```

Stores the room type.

```
booking["days"] = days
```

Stores the number of days.

```
booking["total"] = total
```

Stores the total bill.

So the booking dictionary now contains the complete booking.

22. Booking confirmation

```
print("\nRoom booked successfully!")
```

Displays a success message.

```
print("Room Type:", suggested_type)
```

Displays the booked room type.

```
print("Total Bill: ₹", total)
```

Displays the total amount.

23. If room is already booked

```
else: print("\nRoom is already booked!.")
```

If the selected room's status is "Booked", the customer is informed that the room cannot be booked.

24. If wrong room type is selected

```
else: print("Please select a", suggested_type, "room.")
```

If the customer selects a room that doesn't match the suggested type, the system displays a message.

Example:

Suggested: Double

Customer selects: Single

The system says:

Please select a Double room.

25. If room number doesn't exist

```
else: print("Invalid room number.")
```

If the customer enters something like:

999

the system says the room number is invalid.

26. Option 3 — View Booking

```
elif choice == 3:
```

Runs when the user selects **View Booking**.

if booking:

Checks whether the booking dictionary contains information.

If it contains data, the booking exists.

```
print("\n===== BOOKING DETAILS =====")
```

Displays the booking heading.

Then:

```
print("Customer Name:", booking["name"])
```

Displays the customer name.

```
print("Phone:", booking["phone"])
```

Displays the phone number.

```
print("Number of Members:", booking["members"])
```

Displays the number of members.

```
print("Room Number:", booking["room"])
```

Displays the room number.

```
print("Room Type:", booking["room_type"])
```

Displays the room type.

```
print("Number of Days:", booking["days"])
```

Displays the number of days.

```
print("Total Bill: ₹", booking["total"])
```

Displays the total bill.

If there is no booking:

```
else: print("No booking found.")
```

The system tells the user that there is no booking.

27. Option 4 — Cancel Booking

```
elif choice == 4:
```

Runs when the user selects **Cancel Booking**.

if booking:

Checks whether a booking exists.

If there is a booking:

```
room_no = int(input("Enter the room number to cancel: "))
```

The customer enters the room number they want to cancel.

28. Checking the room number

```
if room_no == booking["room"]:
```

Compares the entered room number with the actual booked room number.

29. Displaying cancellation details

```
print("\n===== CANCELLATION DETAILS =====")
```

Displays the cancellation heading.

```
print("Customer Name:", booking["name"])
```

Displays the customer's name.

```
print("Mobile Number:", booking["phone"])
```

Displays the customer's phone number.

```
print("Room Number:", booking["room"])
```

Displays the room being cancelled.

30. Making the room available again

```
rooms[room_no][2] = "Available"
```

Changes the room status:

Booked → Available

Now another customer can book that room.

31. Removing booking information

```
booking.clear()
```

.clear() removes all information from the booking dictionary.

32. Wrong room number during cancellation

else:

Runs if the customer enters a room number that is not their assigned room.

```
print("Invalid room number! Please enter your assigned room number.")
```

Displays an error message.

33. No booking to cancel

```
else: print("No booking found.")
```

If there is no booking, the system tells the user there is nothing to cancel.

34. Option 5 — Exit

```
elif choice == 5:
```

Runs when the user selects 5.

```
print("THANK YOU for using FAMILY HOTEL ROOM BOOKING!")
```

Displays the final message.

```
break
```

This is extremely important.

It stops the **while True loop**, which ends the program.

35. Invalid menu choice

else:

Runs if the user enters something other than:

22. FINAL CONCLUSION

The Family Hotel Room Booking System successfully provides a simple and efficient solution for managing basic hotel room booking operations using Python. The system allows users to view room availability, select a suitable room based on the number of members, enter customer details, specify the number of days, calculate the total bill, view booking information, and cancel an existing booking.

The project uses fundamental Python programming concepts such as dictionaries, lists, loops, conditional statements, user input, input validation, and data handling. The system automatically updates the room status from Available to Booked when a room is reserved and changes it back to Available when the booking is cancelled.

This project helps reduce manual work and minimizes errors in basic hotel booking operations. It also demonstrates how Python can be applied to solve a real-world problem through a simple console-based application.

The project successfully demonstrates a menu-driven Family Hotel Room Booking System using Python. It combines room management, customer input, validation, room suggestion, booking, billing, viewing, and cancellation in a single application. The project can be further developed into a complete hotel management system by adding permanent storage, multiple bookings, authentication, payments, and a graphical interface.